

WEALINS LLM Benchmark

Veille technologique automatisée sur les modèles de langage
appliquée à l'assurance vie luxembourgeoise

Aristide Yendoulan PANGOU

Stage WEALINS / Foyer Group, Luxembourg

Juillet 2026

Plan

- 1 Contexte et problématique
- 2 La solution : benchmark automatisé
- 3 Les 7 modèles testés
- 4 Méthodologie détaillée
- 5 Résultats et automatisation
- 6 Utilisation et perspectives
- 7 Conclusion

Plan

- 1 Contexte et problématique
- 2 La solution : benchmark automatisé
- 3 Les 7 modèles testés
- 4 Méthodologie détaillée
- 5 Résultats et automatisation
- 6 Utilisation et perspectives
- 7 Conclusion

Un marché des LLM en pleine explosion

Contexte

Les modèles de langage (LLM) se multiplient à grande vitesse. Claude, GPT, Gemini, Llama, Mistral, Grok... de nouveaux modèles sortent chaque semaine.

Le problème concret

Quel LLM utiliser pour nos besoins métier ?

Impossible de les tester manuellement un par un. Les performances varient selon le domaine. Un bon modèle aujourd'hui peut être dépassé demain.

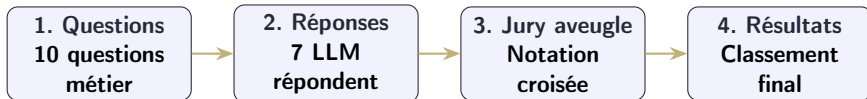
Ce dont on a besoin

Un système qui compare les LLM de façon **objective**, sur nos propres questions métier, et qui se **met à jour automatiquement** sans intervention humaine.

Plan

- 1 Contexte et problématique
- 2 La solution : benchmark automatisé
- 3 Les 7 modèles testés
- 4 Méthodologie détaillée
- 5 Résultats et automatisation
- 6 Utilisation et perspectives
- 7 Conclusion

Vue d'ensemble du système



Collecte
70 réponses collectées
(7 LLM × 10 questions)

Évaluation
70 évaluations croisées
par run
(7 juges × 10 questions)
Soit 350 évaluations sur
5 runs

**Durée &
Robustesse**
5 runs × 45 min
≈ 4 heures au total
Moyenne sur 5 runs

Plan

- 1 Contexte et problématique
- 2 La solution : benchmark automatisé
- 3 Les 7 modèles testés**
- 4 Méthodologie détaillée
- 5 Résultats et automatisation
- 6 Utilisation et perspectives
- 7 Conclusion

Les 7 LLM sélectionnés

Nom	Fournisseur	Modèle
Copilot (GPT)	OpenAI / Microsoft	gpt-5.6-luna-pro
Claude	Anthropic	claude-opus-5-fast
Gemini	Google	gemini-3.6-flash
Llama	Meta	llama-4-maverick
Mistral	Mistral AI	mistral-medium-3-5
DeepSeek	DeepSeek	deepseek-v4-flash-0731
Grok	xAI	grok-4.5

Critère de sélection

Chaque modèle a passé un test de fiabilité avant intégration, dans **les deux rôles** : répondant et juge (7/7 validés).

Couverture du marché

7 fournisseurs différents (OpenAI, Anthropic, Google, Meta, Mistral AI, DeepSeek, xAI) pour une vue représentative.

Plan

- 1 Contexte et problématique
- 2 La solution : benchmark automatisé
- 3 Les 7 modèles testés
- 4 Méthodologie détaillée
- 5 Résultats et automatisation
- 6 Utilisation et perspectives
- 7 Conclusion

Étape 1 Les questions métier

10 vraies questions métier fournies par les équipes WEALINS

- C Alerte client favoritisme
- C Déclaration comptes Hong Kong
- J Directive IDD divergences France
- J Règlement SFDR divergences Belgique
- F Fonds evergreen assurance
- F Rétrocessions upfront fees
- W Don valeurs mobilières Italie
- W Co-souscription 990I ou 757B
- W Donation Hong Kong
- W Résident Thaïlande police luxembourgeoise

Provenance

Questions fournies par les équipes Compliance, Juridique, Finance et WAP de WEALINS.

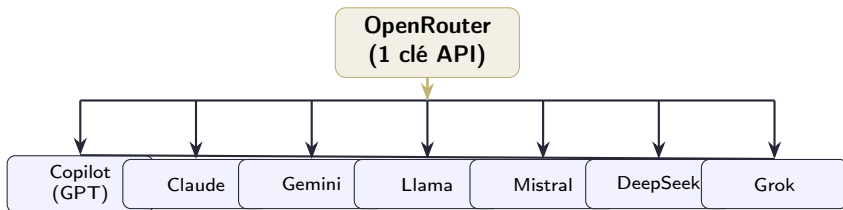
Modulable

Ces questions peuvent être modifiées à tout moment dans le fichier `questions.py`.

Étape 2 Collecte des réponses via OpenRouter

OpenRouter : une passerelle unique vers tous les LLM

Au lieu d'avoir une clé API par fournisseur (Anthropic, OpenAI, Google. . .), on utilise **une seule clé OpenRouter** qui donne accès à tous les modèles simultanément.



Pourquoi « Copilot (GPT) » ?

Copilot n'est pas un LLM

Copilot est un **assistant** intégré dans des outils (VS Code, Microsoft 365). Il n'a pas d'intelligence propre : à chaque requête, il fait appel au modèle **GPT** d'OpenAI en arrière-plan.

L'analogie

GPT = le moteur de la voiture

Copilot = la voiture, avec son tableau de bord adapté à un usage précis (le code).

Même moteur, habillage différent.

Notre choix

Nous testons directement le moteur (GPT), d'où l'appellation **Copilot (GPT)** dans le benchmark : le nom Copilot apparaît, la précision technique est conservée.

Étape 3 Le jury en aveugle

Comment ça fonctionne

- 1 Les réponses des 7 LLM sont **anonymisées**
Claude → "A", GPT → "B"...
- 2 Chaque LLM reçoit les 7 réponses anonymes et **attribue une note** sur 8 critères
- 3 Un LLM peut évaluer sa propre réponse **sans le savoir**
- 4 À la fin, on lève l'anonymat et on calcule les scores finaux

Pourquoi en aveugle ?

Pour éviter tout biais. Si un LLM savait quelle réponse lui appartient, il pourrait se favoriser ou défavoriser un concurrent. L'anonymisation garantit une évaluation neutre.

Résultat

70 évaluations croisées par run
(7 juges × 10 questions)

Étape 4 Calcul des scores pondérés

Critère	Poids
Exactitude technique	20 %
Vocabulaire assurance	20 %
Pertinence réglementaire	15 %
Complétude	15 %
Clarté et lisibilité	10 %
Absence d'hallucinations	10 %
Applicabilité pratique	5 %
Gestion de l'incertitude	5 %

Formule

Chaque réponse est notée de 0 à 10 sur les 8 critères par chacun des 7 juges. Le score final est la **moyenne pondérée** de toutes les notes reçues.

Note importante

Ces pondérations sont une **proposition initiale**. Elles peuvent être ajustées par l'équipe métier selon les priorités de WEALINS.

Tous les modèles ne peuvent pas être juges

Un phénomène documenté par la recherche

Les études académiques sur le *LLM-as-a-judge* (MT-Bench, Chatbot Arena) montrent qu'aucun modèle n'est fiable à 100 % comme évaluateur même GPT ne s'aligne sur le jugement humain qu'à environ 80 %.

Défaillances observées

- Réponses vides (mode « réflexion » qui épuise le budget de tokens)
- Non-respect du format de notation demandé
- Refus de noter (« je ne peux pas juger »)

Réf. : Zheng et al. (2023), *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*, NeurIPS.

Notre réponse méthodologique

Test systématique avant intégration : chaque modèle candidat doit réussir un test dans les deux rôles (répondant et juge). Seuls les modèles fiables entrent au benchmark ; les autres sont réévalués à chaque nouvelle version.

Plan

- 1 Contexte et problématique
- 2 La solution : benchmark automatisé
- 3 Les 7 modèles testés
- 4 Méthodologie détaillée
- 5 Résultats et automatisation**
- 6 Utilisation et perspectives
- 7 Conclusion

Résultats sur des questions métier WEALINS (07 Août 2026)

Rang	LLM	Score /10	Moyenne de 5 runs
1	Claude	8.95	
2	Copilot (GPT)	8.61	
3	Grok	8.3	
4	Gemini	8.04	
5	DeepSeek	6.13	
6	Llama	5.65	
7	Mistral	5.15	

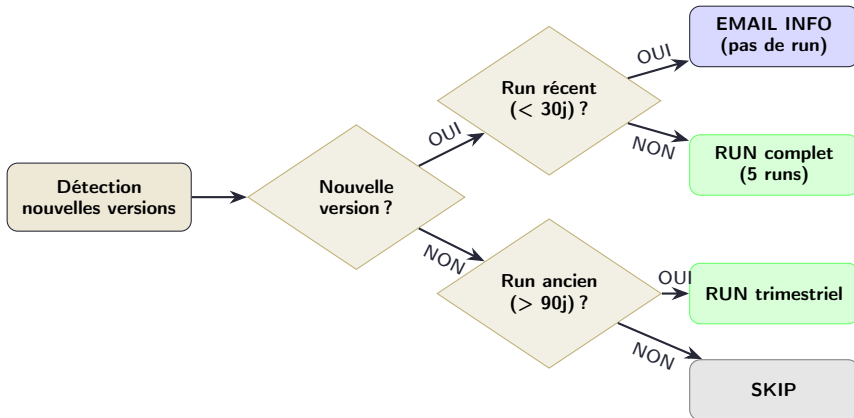
Observation

Deuxième benchmark sur de vraies questions métier WEALINS (Compliance, Juridique, Finance, WAP). Les scores ont significativement progressé avec les nouveaux modèles, preuve que la sélection automatique fonctionne.

À noter

Claude (Anthropic) s'impose en tête avec 8.95/10.
GPT et Grok complètent le podium.
Mistral et Llama reculent nettement.

Logique de décision automatique



Les notifications email

Trois types de notifications automatiques

Email 1 Info

Envoyé quand une nouvelle version est détectée **mais** le dernier run date de moins de 30 jours.

Contient les nouvelles versions disponibles.

Aucun run lancé.

Email 2 Avant run

Envoyé quand un run est déclenché. Contient :

- Nouvelles versions détectées
- Modèles **mis à jour**
- Modèles **conservés** (instable)

Email 3 Après run

Envoyé à la fin du benchmark. Contient :

- Classement final des 7 LLM
- Rappel des mises à jour
- Lien vers le dashboard

Zéro action manuelle

Le système décide, teste, met à jour, benchmark, notifie tout automatiquement. Maximum **1 run par mois**, même si de nouvelles versions sortent chaque semaine.

Automatisation complète

Ce qui se passe automatiquement

- ➊ **Détection** des nouvelles versions de LLM
- ➋ **Test & mise à jour** automatique des modèles fiables
- ➌ **Décision** : run nécessaire ou pas ?
- ➍ **Exécution** du benchmark (5 runs $\times$ 45 min $\approx$ 4h)
- ➎ **Notification** par email
- ➏ **Sauvegarde** des résultats sur GitHub
- ➐ **Mise à jour** du dashboard en ligne

Déclenchement

Automatique : tous les lundis à 4h UTC (6h Luxembourg)

Le système applique la logique :

- Nouvelle version + run récent ($< 30j$) $\rightarrow$ email seul
- Nouvelle version + run ancien ($> 30j$) $\rightarrow$ run complet
- Aucune nouveauté + $> 90j$ $\rightarrow$ run trimestriel

Manuel : un clic sur GitHub Actions suffit pour forcer un run à tout moment.

Dashboard public

Accessible depuis n'importe quel navigateur, sans installation.

Mis à jour automatiquement après chaque run.

`wealins-benchmark-ufwn2...streamlit.app`

Plan

- 1 Contexte et problématique
- 2 La solution : benchmark automatisé
- 3 Les 7 modèles testés
- 4 Méthodologie détaillée
- 5 Résultats et automatisation
- 6 Utilisation et perspectives**
- 7 Conclusion

Comment l'utiliser au quotidien

Consulter les résultats

Ouvrir le lien dashboard dans un navigateur. Aucune installation. Mis à jour automatiquement.

→ **Accessible à toute l'équipe**

Adapter le benchmark

Modifier les questions, les critères, ou ajouter de nouveaux LLM dans le code.

→ **Personnalisable selon les besoins**

Lancer un run manuellement

Aller sur GitHub → onglet Actions → cliquer *Run workflow*.

→ **Résultats en (5 runs × 45 min ≈ 4h)**

À venir

Script d'installation en un double-clic pour lancer le projet sur n'importe quel PC du bureau sans toucher au code.

Plan

- 1 Contexte et problématique
- 2 La solution : benchmark automatisé
- 3 Les 7 modèles testés
- 4 Méthodologie détaillée
- 5 Résultats et automatisation
- 6 Utilisation et perspectives
- 7 Conclusion**

Conclusion

Ce que le projet apporte à WEALINS

Un outil de veille LLM **automatisé, objectif et spécifique** au domaine de l'assurance vie luxembourgeoise. Zéro intervention manuelle après configuration initiale.

Points forts

Benchmark sur nos propres questions métier.

Jury en aveugle → résultats objectifs.

Historique d'évolution dans le temps.

Dashboard accessible sans installation.

Notifications email automatiques.

Merci pour votre attention.

Questions ?

Annexes techniques

Détails techniques disponibles

Ces slides sont là pour répondre aux questions techniques.

Fichiers principaux

A1 questions.py
A2 llm_clients.py
A3 evaluator.py
A4 scheduler.py
A5 run.py
A6 dashboard.py
A7 benchmark.yml
A8 requirements.txt

Fichiers support

B1 benchmark.py
B2 detector.py
B3 updater.py
B4 notifier.py
B5 loader.py
B6 utils.py

Dossier résultats

C1 results/

A1 questions.py : cœur métier du système

Structure d'une question

```
{
  "id": 1,
  "categorie": "Compliance",
  "titre": "Alerte client",
  "question": "..."}

```

Garde-fou automatique

```
total = sum(
  c["poids"]
  for c in CRITERES.values()
)
assert round(total, 2) == 1.0
```

Si les poids ne font pas 100% → le système refuse de démarrer.

Importé par 3 fichiers

benchmark.py → envoie les questions
evaluator.py → note les réponses
dashboard.py → affiche les résultats

Critère	Poids
Exactitude technique	20 %
Vocabulaire assurance	20 %
Pertinence réglementaire	15 %
Complétude	15 %
Clarté et lisibilité	10 %
Absence hallucinations	10 %
Applicabilité pratique	5 %
Gestion incertitude	5 %

A2 llm_clients.py : appel des 7 LLM

Une seule fonction d'appel

```
def appeler_llm(llm_id, prompt):  
    config = LLM_CONFIG[llm_id]  
    modele = config["modele"]  
    time.sleep(1)  
    return _appeler_openrouter(  
        modele, prompt  
    )
```

Le secret : SDK OpenAI + OpenRouter

```
client = OpenAI(  
    api_key=OPENROUTER_API_KEY,  
    base_url="https://openrouter  
    .ai/api/v1"  
)
```

Pourquoi ce design ?

1 seule clé API pour 7 fournisseurs

1 seule fonction appeler_llm()
appelée partout dans le projet

Ajouter un LLM = ajouter
1 entrée dans LLM_CONFIG

Choix techniques

temperature=0 → résultats
reproductibles à iso-conditions

time.sleep(1) → respecte
les rate limits OpenRouter

timeout=60 → évite
de bloquer le run entier

A3 evaluator.py : le jury en aveugle

3 étapes bien séparées

1. `construire_prompt_notation()`
→ prépare la question + réponses anonymes
2. `jury_tournant()`
→ chaque LLM note les autres
3. `calculer_scores_finaux()`
→ agrège et pondère toutes les notes

Le jury ne sait pas qui il est

Le LLM reçoit les réponses anonymisées A, B, C... sans savoir lequel est le sien.

→ Évaluation **neutre et objective**

Un LLM peut noter sa propre réponse **sans le savoir** c'est voulu.

Fallback intelligent

```
notes = parser_notes_texte(reponse)
if not notes:
    notes = parser_json_llm(reponse)
```

Si le LLM ne respecte pas le format → on essaie autrement.

Garde-fous

`nettoyer_note()` → force toutes les notes entre 0 et 10

`time.sleep(2)` → respecte les rate limits entre chaque juge

`if c in CRITERES` → ignore les critères non reconnus

A4 scheduler.py : décision automatique

4 cas de décision

1. Nouvelle version + run > 30j
→ **RUN complet**
2. Nouvelle version + run < 30j
→ **EMAIL** seul (pas de run)
3. Aucune nouveauté + run > 90j
→ **RUN trimestriel**
4. Aucune nouveauté + run récent
→ **SKIP**

Constantes facilement modifiables

DELAI_MIN_JOURS = 30

DELAI_MAX_JOURS = 90

Toute la logique repose sur ces 2 valeurs.
Changer le délai = modifier **une seule ligne**.

Astuce si pas d'historique

```
if derniere_date is None:
    jours_ecoules = 999
```

Aucun run trouvé → on considère le système comme très ancien
→ run lancé automatiquement.

Retour tracé

Chaque décision retourne une raison explicite :

```
nouvelle_version_detectee
run_trimestriel
rien_a_signaler
```

→ Traçabilité complète dans les logs GitHub Actions

A5 run.py : chef d'orchestre

Pipeline complet

1. `decider_run()`
→ run nécessaire ou pas ?
2. `evaluer_et_mettre_a_jour()`
→ si nouvelles versions détectées
3. `lancer_benchmark()`
→ les 5 runs × 45 min
4. `notifier_resultats()`
→ email avec classement final

2 modes d'utilisation

```
# GitHub Actions (automatique)
run_avec_decision()

# Test manuel
if __name__ == "__main__":
    lancer_benchmark()
```

Principe de conception

`run.py` ne fait **aucune logique** lui-même.

Il **importe et coordonne** :

`scheduler` → décision
`updater` → mise à jour
`benchmark` → exécution
`notifier` → emails
`loader` → données

→ Chaque module est **indépendant et testable** séparément.

Rapport unifié

Si nouvelles versions détectées,
 le rapport de mise à jour est **inclus dans l'email résultats**
 → une seule notification complète.

A6 dashboard.py : interface Streamlit

4 onglets interactifs

1. Classement général
→ podium + barres + tableau
2. Par critère
→ radar chart + heatmap
3. Réponses complètes
→ lecture par question/LLM
4. Évolution temporelle
→ courbes run par run

Cache intelligent

```
@st.cache_data(ttl=300)
def charger_donnees():
    ...
```

Les données sont rechargées maximum toutes les 5 minutes.
→ Pas de ralentissement à chaque clic utilisateur.

Stack de visualisation

plotly → barres horizontales,
radar chart, heatmap,
courbes d'évolution

pandas → tableaux interactifs

streamlit → interface web
sans HTML/CSS complexe

Robustesse

Si aucune donnée disponible
→ message d'attente clair
→ affiche quand même les
questions configurées

Couleurs des LLM viennent
directement de llm_clients.py
→ **cohérence visuelle** garantie
sur tout le dashboard

B1 benchmark.py : chef d'orchestre

4 phases par run

1. `collecter_reponses()`
→ 10 questions × 7 LLM
2. `pseudonymiser_reponses()`
→ A, B, C... mapping secret
3. `jury_tournant()`
→ chaque LLM note les autres
4. `calculer_scores_finaux()`
→ moyenne pondérée

Robustesse statistique

NB_RUNS = 5

Le benchmark tourne **5 fois**.
Le score final = moyenne des 5 runs.
→ Élimine les variations aléatoires
d'un seul appel API.

Prompt calibré

"Sois concis et précis,
vise 250-300 mots."

Toutes les réponses ont
la même contrainte de longueur.
→ Comparaison à **iso-conditions**
entre tous les LLM.

Aucune logique propre

`benchmark.py` ne fait que
coordonner les autres modules :

`llm_clients` → appels API
`utils` → pseudonymisation
`evaluator` → jury + scores
`loader` → sauvegarde

→ Architecture **modulaire**
chaque module testable séparément.

A7 benchmark.yml : GitHub Actions

Déclenchement

```
on:
  schedule:
    - cron: '0 4 * * 1'
  workflow_dispatch:
```

Automatique : tous les lundis
à 4h UTC (6h Luxembourg)

Manuel : bouton *Run workflow*
dans l'onglet Actions de GitHub

Garde-fous

timeout-minutes: 360
→ le run s'arrête après 6h maximum
évite un run infini qui consomme
des crédits API indéfiniment

[skip ci] dans le message commit
→ évite une **boucle infinie**
(le commit ne redéclenche pas
un nouveau workflow)

Sécurité des clés API

```
env:
  OPENROUTER_API_KEY:
    ${{ secrets.OPENROUTER_API_KEY }}
```

Jamais dans le code.
Stockées dans **GitHub Secrets**.
Injectées uniquement à l'exécution.

Ce qui est sauvegardé

```
git add results/
      src/llm_clients.py
```

results/ → les scores JSON
llm_clients.py → mis à jour
automatiquement si nouvelles
versions détectées
→ Streamlit se met à jour seul

A8 requirements.txt : dépendances

7 packages seulement

```
openai>=1.40.0  
streamlit>=1.37.0  
plotly>=5.22.0  
pandas>=2.2.0  
python-dotenv>=1.0.0  
requests>=2.32.0  
notebook>=7.0.0
```

Rôle de chaque package

openai → SDK pour appeler
OpenRouter (pas OpenAI direct !)

streamlit → dashboard web

plotly → graphiques interactifs

pandas → tableaux de données

python-dotenv → charge le .env

requests → appels HTTP directs

notebook → exploration données

Point fort

7 packages pour un système complet : collecte, évaluation, automatisation, dashboard.

→ Stack **légère et maintenable**.

Installation en **1 commande** :

```
pip install -r requirements.txt
```

B1 detector / updater / loader

detector.py

Interroge l'API OpenRouter pour lister tous les modèles disponibles

Compare les **dates de sortie** avec les modèles actuels

Filtre les modèles inutiles :
guard, embedding,
tts, image...

→ Détecte **automatiquement** toute nouvelle version

updater.py

Teste chaque nouveau modèle dans les **2 rôles** :
répondant et juge

Si fiable → met à jour
llm_clients.py
avec un simple replace()

Commit + push automatique sur GitHub

→ **Zéro intervention** humaine nécessaire

loader.py

Point d'entrée **unique** pour toutes les données

Chemins définis **une seule fois**
Si dossier déplacé → 1 ligne

```
def _mettre_a_jour...
```

_ = convention Python pour fonction **privée**

→ Historique reconstruit depuis les JSON existants sans conflit Git

B2 utils.py & notifier.py

utils.py boîte à outils

Pseudonymisation aléatoire

`random.shuffle()` → ordre différent
à chaque run

Calcul pondéré normalisé

fonctionne même si poids $\neq 1.0$

`nettoyer_note()` → corrige

automatiquement les notes invalides :
15 → 10.0 / -2 → 0.0 / "abc" → 0.0

Double parser avec 4 fallbacks :

JSON propre → “json → “

→ recherche {} n'importe où

→ Aucune réponse LLM perdue

notifier.py emails

Gmail SMTP natif Python

aucune librairie externe
juste `smtplib` (standard)

```
with smtplib.SMTP(
    "smtp.gmail.com", 587) as s:
    s.starttls()
    s.login(...)
    s.send_message(msg)
```

Mot de passe d'application

pas le vrai mot de passe Gmail

→ généré depuis Google Security

2 emails distincts :

nouvelles versions détectées
résultats du run terminé

C1 results/ : stockage des données

Structure du dossier

```
results/  
|-- benchmark_2026-07-19.json  
|-- benchmark_2026-07-21.json  
|-- benchmark_2026-08-03.json  
|-- benchmark_2026-08-07.json  
\-- historique.json
```

1 fichier JSON par run
nommé avec la date du jour
→ tri automatique par date

Contenu d'un fichier

date → date du run

scores → score global
+ scores par critère
pour chaque LLM

reponses → texte complet
de chaque réponse

mapping_revele → qui
était A, B, C...

nb_runs → nombre de runs
moyennés

Pas de base de données

Tout est en **JSON plat**.
Pas de SQL, pas de serveur.

- Lisible par n'importe qui
- Versionné sur GitHub
- Zéro infrastructure

Historique intelligent

loader.py reconstruit
l'historique **dynamiquement**
depuis les fichiers existants

- Pas de conflit Git
- Ajout d'un run = ajout
d'un fichier JSON